

Drive247 — Xero & Zoho Books Finance Sync

Developer Implementation Guide

*This document is the single source of truth for the Finance Sync feature. It covers product goals, architecture, database schema, OAuth flows, edge functions, portal UI, mapping logic, error handling, and the phased build sequence — for both **Xero** and **Zoho Books**.*

Table of Contents

1. [Product Summary](#)
 2. [Architecture Principles](#)
 3. [Xero vs Zoho — Side-by-Side](#)
 4. [Developer Onboarding](#)
 5. [Database Schema](#)
 6. [OAuth Connection Flow](#)
 7. [Provider Abstraction Layer](#)
 8. [Event-to-Provider Mapping](#)
 9. [Edge Functions Reference](#)
 10. [Portal UI — Screens & Specifications](#)
 11. [Vehicle Profitability Dashboard](#)
 12. [Historical Backfill](#)
 13. [Account & Tax Mapping](#)
 14. [Sync Status, Logs & Error Handling](#)
 15. [Webhooks \(Phase 2\)](#)
 16. [Testing Strategy](#)
 17. [Phased Build Sequence](#)
 18. [Environment Variables](#)
 19. [API Reference Cheat Sheet](#)
 20. [Appendix — Glossary & Useful Links](#)
-

1. Product Summary

1.1 What we are building

A **two-provider Finance Sync module** inside Drive247 that automatically pushes every rental-related financial event (invoice, payment, refund, deposit, damage charge, extension, etc.) into the tenant's accounting system of choice — **Xero** or **Zoho Books**.

1.2 Why we are building it

- Operators with 15+ cars currently spend **6–10 hours per month** manually re-keying Drive247 data into their accountant's system.
- Drive247 already holds the structured rental data (vehicle, customer, dates, charges) that the accounting system needs.
- Once invoices flow automatically, Drive247 becomes the operational hub *plus* the reconciled financial source — a major stickiness and pricing lever.
- The **Vehicle Profitability Dashboard** (built on the same internal ledger) is something neither Xero nor Zoho can offer, because they don't understand the rental domain.

1.3 Strategic positioning

*Drive247 is **not** an accounting system. It is the operational layer that feeds the accounting system.*

We never want a tenant creating invoices in two places. **Drive247 is the only place financial events are born.** Xero/Zoho receive a clean, deduplicated stream of those events.

1.4 Provider priority

Order	Provider	Reason
1	Xero	Cleanest API, premium positioning, best for 20+ car operators with bookkeepers
2	Zoho Books	Active prospect on Zoho; cost-conscious operator segment
3	QuickBooks Online	US market expansion (not in scope for v1)

1.5 In scope (MVP)

- `financial_events` internal ledger
- Per-tenant OAuth connection to Xero **and** Zoho Books
- One-way push sync: contacts, invoices, payments, credit notes, refunds, captured deposits
- Account & tax mapping UI in Settings → Integrations → Accounting
- Per-invoice sync status badges + Sync Log page
- Historical backfill (sync existing rentals retroactively)
- Vehicle Profitability Dashboard
- Tax/VAT line-item handling (non-negotiable for invoice APIs)

1.6 Out of scope (Phase 2+)

- Two-way sync (reading paid status back from accounting system)
 - Webhook ingestion (Xero/Zoho → Drive247)
 - Per-vehicle Xero tracking categories
 - Stripe payout → bank reconciliation
 - Partner/co-host payouts as bills
 - QuickBooks Online integration
 - Multi-currency tenants
-

2. Architecture Principles

These five principles drive every implementation decision. Re-read them before designing any new piece of this module.

2.1 Drive247 is the source of truth

Every financial event originates in Drive247. The sync layer is **one-way push only** in MVP. We never let an operator type an invoice directly into Xero/Zoho and expect Drive247 to reconcile.

2.2 Provider-agnostic from day one

Even though Xero ships first, the abstraction must be designed so Zoho slots in without rewriting business logic. The sync layer talks to an **AccountingProvider interface**, never to Xero/Zoho SDKs directly.

2.3 `financial_events` is the bridge

Existing tables (`rental_charges` , `payments` , `rental_extensions` , etc.) **stay untouched**. A new `financial_events` table is written-to in the same code paths, and is the **only** thing the sync layer reads. This decouples accounting concerns from operational data.

2.4 Idempotent, eventually consistent

Every sync operation must be idempotent. We rely on:

- A **unique external reference** stored on `financial_events` per provider.
- A **state machine** (`pending` → `syncing` → `synced` / `failed`) per event-provider pair.
- **Exponential backoff** retries (1m, 5m, 30m, 2h, 12h, dead-letter).

2.5 Tenant-isolated everything

Every OAuth token, every sync job, every log entry is namespaced by `tenant_id` . The same code path serves UK tenants on Xero and US tenants on Zoho; nothing crosses tenant boundaries.

3. Xero vs Zoho — Side-by-Side

Concern	Xero	Zoho Books
Auth	OAuth 2.0 (PKCE supported)	OAuth 2.0
Token expiry	Access: 30 min · Refresh: 60 days	Access: 1 hr · Refresh: never (unless revoked)
Refresh strategy	Refresh token rotates on every refresh — must persist new one	Refresh token is stable
Data centres	Single global	Multi-region: <code>.com</code> , <code>.eu</code> , <code>.in</code> , <code>.com.au</code> , <code>.jp</code> , <code>.sa</code>
Org identifier	<code>tenantId</code> returned in token exchange	<code>organization_id</code> (must be queried separately)
Rate limits	60 calls/min, 5,000/day per tenant	100 calls/min per org, daily quota varies by plan
Contacts API	<code>POST /Contacts</code>	<code>POST /contacts</code>
Invoices API	<code>POST /Invoices</code>	<code>POST /invoices</code>
Payments API	<code>POST /Payments</code>	<code>POST /customerpayments</code>
Credit notes	<code>POST /CreditNotes</code>	<code>POST /creditnotes</code>
Webhooks	Centralised, signed with <code>x-xero-signature</code>	Per-org workflow webhooks, configured inside Zoho Books
Tax handling	<code>TaxType</code> codes (e.g. <code>OUTPUT2</code> for UK 20% VAT)	<code>tax_id</code> (UUID, must be fetched per org)
Tracking categories	Native first-class concept	Custom fields / tags

Sandbox	Demo Company (free, auto-provisioned)	No true sandbox — use a free Books org
Marketplace	Required (and certification) for >25 connected orgs	Not required
SDK	Official Node.js SDK available	No official Deno SDK — use raw HTTP

Implication: the abstraction has to handle two structurally different worlds — especially around tax codes (Xero's enum vs Zoho's per-org UUIDs) and refresh token rotation.

4. Developer Onboarding

*The developer building this module should set up **their own** Xero and Zoho developer accounts using their personal/work email. Drive247 will supply the **production** API credentials separately once the module is ready to go live.*

4.1 Xero developer setup (developer to complete)

1. Sign up at <https://developer.xero.com/> using your work email.
2. Create a free **Demo Company** — Xero auto-provisions one on first login.
3. Go to **My Apps → New App**:
 - App name: Drive247 Dev — <your name>
 - Integration type: **Web app**
 - Company URL: <https://drive-247.com>
 - Redirect URI: <http://localhost:54321/functions/v1/xero-oauth-callback> (local Supabase)
4. Copy the **Client ID** and **Client Secret** into your local `.env` :

```
XERO_CLIENT_ID=...
XERO_CLIENT_SECRET=...
XERO_REDIRECT_URI=http://localhost:54321/functions/v1/xero-oauth-callback
```

5. Required scopes (request all of these in the OAuth URL):
 - openid profile email
 - accounting.contacts
 - accounting.transactions
 - accounting.settings
 - accounting.journals.read
 - offline_access (mandatory for refresh tokens)
6. Confirm you can complete a manual OAuth round-trip using Xero's API Explorer.

4.2 Zoho developer setup (developer to complete)

1. Sign up at <https://api-console.zoho.com/> using your work email.
2. Create a free **Zoho Books trial organisation** at <https://books.zoho.com/> (14-day trial is sufficient; you can also use the free Indian-edition tier).
3. In the API Console, click **Add Client → Server-based Applications**:
 - Client Name: Drive247 Dev — <your name>
 - Homepage URL: <https://drive-247.com>

- o Authorized Redirect URI: `http://localhost:54321/functions/v1/zoho-oauth-callback`

4. Copy the **Client ID** and **Client Secret** into your local `.env` :

```
ZOHO_CLIENT_ID=...
ZOHO_CLIENT_SECRET=...
ZOHO_REDIRECT_URI=http://localhost:54321/functions/v1/zoho-oauth-callback
ZOHO_DC=com # default data centre for dev
```

5. Required scopes:

- o `ZohoBooks.contacts.ALL`
- o `ZohoBooks.invoices.ALL`
- o `ZohoBooks.customerpayments.ALL`
- o `ZohoBooks.creditnotes.ALL`
- o `ZohoBooks.settings.READ`

6. Confirm you can complete a manual OAuth round-trip using `accounts.zoho.com/oauth/v2/auth?...`

4.3 Provided test accounts (use at OAuth login screen)

Drive247 has provisioned a real Xero organisation and a real Zoho (Finance Plus / Books) organisation for you to connect to during development and QA. **These are distinct from your developer account in 4.1 / 4.2:**

- **Your developer account** (sections 4.1 / 4.2) = where you registered the OAuth app and obtained `CLIENT_ID` and `CLIENT_SECRET` .
- **These provided accounts** = the actual Xero / Zoho organisation that Drive247 will sync invoices, contacts, payments and refunds into when you test the "Connect Xero" / "Connect Zoho" flow inside the portal.

When the OAuth consent screen asks you to sign in, use the credentials below.

Xero

Field	Value
Login URL	https://login.xero.com
Email (username)	neemacortek@gmail.com
Account name	Neemacortek
Password	Cortek321!

Zoho (Finance Plus / Books)

Field	Value
Login URL	https://accounts.zoho.com
Email (username)	neemacortek@gmail.com
Account name	Neemacortek

Password	Cortek2907!
----------	-------------

Security & handling:

- Treat these credentials like production secrets — do **not** commit them to git, post them in tickets, or share them outside the engineering team.
- If you suspect either credential has leaked, notify the Drive247 team immediately so we can rotate.
- Two-factor authentication may be enabled on these accounts. If you hit a 2FA prompt during OAuth, coordinate with the Drive247 team to receive the code.
- The same email (`neemacortek@gmail.com`) is used for both providers — make sure you're on the correct provider's login page before entering the password.

4.4 Drive247 environment prep

```
git checkout -b feat/finance-sync
cp .env.example .env.local
# add Xero + Zoho credentials from steps 4.1 and 4.2

# Bring up local Supabase
npx supabase start

# Apply pending migrations (you'll create new ones for finance-sync)
npx supabase db reset

# Boot the portal
npm run dev:portal
```

4.5 Recommended dev tools

- **Postman / Insomnia** with the Xero and Zoho collections imported (both publish public collections).
- **ngrok** for exposing local webhook endpoints to Xero/Zoho during Phase 2.
- The **Supabase Edge Function debugger** (`npx supabase functions serve --debug`).

5. Database Schema

All tables live in the `public` schema and are protected by RLS. The migration filename convention follows `YYYYMMDDHHMMSS_*.sql` (matching the existing project pattern).

5.1 accounting_connections

One row per tenant per provider — only one **active** connection per provider per tenant.

```
create type accounting_provider as enum ('xero', 'zoho');
create type accounting_connection_status as enum ('active', 'expired', 'revoked',
'error');

create table accounting_connections (
  id                uuid primary key default gen_random_uuid(),
  tenant_id         uuid not null references tenants(id) on delete cascade,
```

```

provider          accounting_provider not null,
status            accounting_connection_status not null default 'active',

-- OAuth state
access_token      text not null,
refresh_token     text not null,
token_expires_at  timestampz not null,

-- Provider-specific identifiers
external_org_id   text not null,          -- Xero tenantId or Zoho
organization_id
external_org_name text,
external_region   text,                  -- Zoho DC: com / eu / in / com.au;
null for Xero

-- Telemetry
last_synced_at    timestampz,
last_error        text,
connected_by      uuid references app_users(id),
connected_at      timestampz not null default now(),
disconnected_at   timestampz,

created_at        timestampz not null default now(),
updated_at        timestampz not null default now()
);

create unique index accounting_connections_active_uniq
on accounting_connections (tenant_id, provider)
where status = 'active';

create index accounting_connections_tenant_idx
on accounting_connections (tenant_id);

```

RLS: tenant members can SELECT their own row (without `access_token` / `refresh_token` — expose via a view). Only `service_role` can INSERT/UPDATE/DELETE.

5.2 financial_events

The internal ledger — every chargeable, refundable, or accountable thing that happens in Drive247.

```

create type financial_event_type as enum (
  'rental_charge',
  'deposit_capture',
  'security_hold_release',
  'insurance_charge',
  'late_fee',
  'mileage_charge',
  'damage_charge',
  'charging_cost',
  'extension_charge',
  'refund',
  'discount',

```

```

    'maintenance_expense',
    'partner_payout'
);

create type financial_event_status as enum ('open', 'finalised', 'voided');

create table financial_events (
    id                uuid primary key default gen_random_uuid(),
    tenant_id         uuid not null references tenants(id) on delete cascade,

    -- What it relates to
    rental_id         uuid references rentals(id),
    customer_id       uuid references customers(id),
    vehicle_id        uuid references vehicles(id),

    -- The money
    event_type        financial_event_type not null,
    amount_cents      integer not null,          -- can be negative for
refunds/discounts
    tax_cents         integer not null default 0,
    currency          text not null,
    occurred_at       timestampz not null,

    -- Lifecycle
    status            financial_event_status not null default 'finalised',
    source_table      text,                    -- e.g. 'rental_charges',
'payments'
    source_id         uuid,

    -- Free-form
    description       text,
    metadata          jsonb not null default '{}'::jsonb,

    created_at        timestampz not null default now(),
    updated_at        timestampz not null default now()
);

create index financial_events_tenant_idx      on financial_events (tenant_id,
occurred_at desc);
create index financial_events_rental_idx      on financial_events (rental_id);
create index financial_events_vehicle_idx     on financial_events (vehicle_id);
create index financial_events_source_idx      on financial_events (source_table,
source_id);

```

5.3 financial_event_sync_state

Tracks the sync state of one `financial_event` against one provider. Two rows per event when both Xero and Zoho are connected.

```

create type sync_state as enum ('pending', 'syncing', 'synced', 'failed',
'skipped');

```



```

create table financial_event_sync_state (
  id                uuid primary key default gen_random_uuid(),
  financial_event_id  uuid not null references financial_events(id) on delete
cascade,
  tenant_id          uuid not null references tenants(id) on delete cascade,
  provider            accounting_provider not null,
  state              sync_state not null default 'pending',

  -- Idempotency keys
  external_invoice_id text,
  external_payment_id text,
  external_credit_note_id text,
  external_contact_id text,

  attempts           integer not null default 0,
  last_attempt_at     timestampz,
  next_attempt_at     timestampz,
  last_error          text,
  last_error_code     text,

  synced_at           timestampz,
  created_at          timestampz not null default now(),
  updated_at          timestampz not null default now()
);

create unique index financial_event_sync_state_uniq
  on financial_event_sync_state (financial_event_id, provider);

create index financial_event_sync_state_pending_idx
  on financial_event_sync_state (state, next_attempt_at)
  where state in ('pending', 'failed');

```

5.4 accounting_account_mappings

Tenant-configured mapping of Drive247 event types → provider account / tax codes.

```

create table accounting_account_mappings (
  id                uuid primary key default gen_random_uuid(),
  tenant_id          uuid not null references tenants(id) on delete cascade,
  provider            accounting_provider not null,
  event_type         financial_event_type not null,

  external_account_code text not null,
  external_account_name text,                -- denormalised for UI
  external_tax_code    text,
  external_tax_rate     numeric(6,3),

  is_default          boolean not null default false,
  created_at          timestampz not null default now(),
  updated_at          timestampz not null default now()
)

```

```
);

create unique index accounting_account_mappings_uniq
  on accounting_account_mappings (tenant_id, provider, event_type);
```

5.5 accounting_contact_links

Tracks which Drive247 customers have already been pushed to which provider, so we don't create duplicates.

```
create table accounting_contact_links (
  id                uuid primary key default gen_random_uuid(),
  tenant_id         uuid not null references tenants(id) on delete cascade,
  customer_id       uuid not null references customers(id) on delete cascade,
  provider          accounting_provider not null,
  external_contact_id text not null,
  created_at        timestampz not null default now()
);

create unique index accounting_contact_links_uniq
  on accounting_contact_links (tenant_id, provider, customer_id);
```

5.6 RLS overview

For every table above, the policy template is:

```
alter table <table> enable row level security;

create policy "<table>_tenant_select"
  on <table> for select
  using (tenant_id = get_user_tenant_id() or is_super_admin());

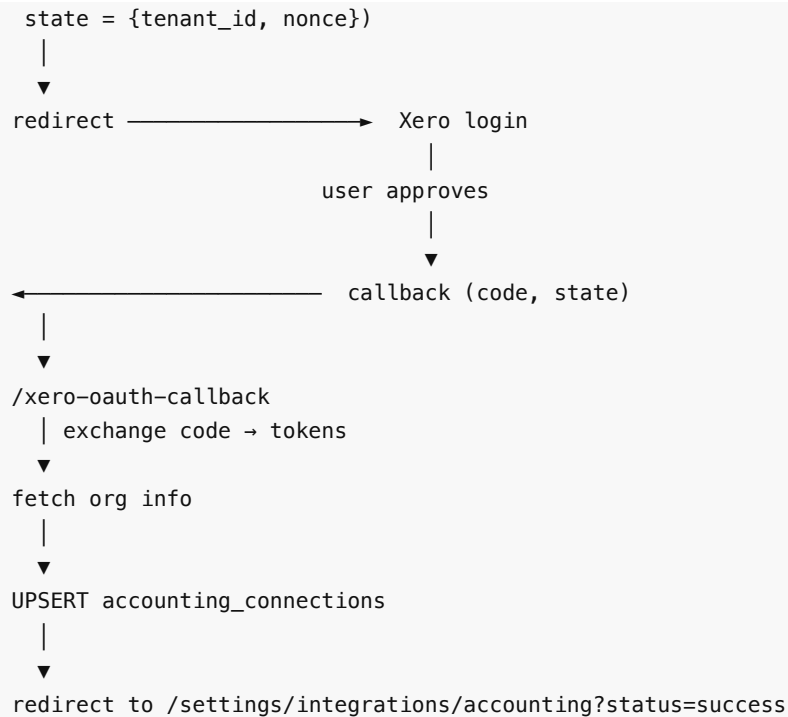
create policy "<table>_service_role_all"
  on <table> for all
  using (auth.role() = 'service_role');
```

OAuth tokens (`access_token` , `refresh_token`) MUST be excluded from any client-facing SELECT — expose `accounting_connections_public` view instead, projecting only safe columns.

6. OAuth Connection Flow

6.1 High-level diagram (both providers)





6.2 Why we use a server-managed state parameter

We never trust the redirect to carry `tenant_id` directly. Instead:

1. `/xero-oauth-start` writes a short-lived row to `oauth_state` (`tenant_id`, `nonce`, `redirect_back`, `expires_at`).
2. The `state` query param is just the nonce.
3. `/xero-oauth-callback` looks up the nonce, validates expiry, then proceeds.

6.3 Token refresh

A scheduled edge function `refresh-accounting-tokens` runs every 10 minutes:

```

select id
from accounting_connections
where status = 'active'
  and token_expires_at < now() + interval '15 minutes';

```

For each row:

- **Xero:** call `https://identity.xero.com/connect/token` with the refresh token, **persist the new refresh token** (Xero rotates).
- **Zoho:** call `https://accounts.zoho.<region>/oauth/v2/token` — refresh token is stable, just update `access_token` and `token_expires_at`.

On failure (3 consecutive 4xx errors), set `status = 'expired'` and notify the tenant via in-app reminder.

6.4 Provider-specific URLs

Step	Xero	Zoho
------	------	------

Authorize URL	https://login.xero.com/identity/connect/authorize	https://accounts.zoho.<region>/oauth/v2/auth
Token URL	https://identity.xero.com/connect/token	https://accounts.zoho.<region>/oauth/v2/token
Org list endpoint	https://api.xero.com/connections (returns tenantId)	https://www.zohoapis.<region>/books/v3/organizations
API base	https://api.xero.com/api.xro/2.0	https://www.zohoapis.<region>/books/v3

For Zoho, <region> defaults to `com` ; the tenant can pick during connect.

7. Provider Abstraction Layer

7.1 The TypeScript interface

```
// supabase/functions/_shared/accounting/types.ts

export interface AccountingProvider {
  readonly name: 'xero' | 'zoho';

  upsertContact(input: ContactInput): Promise<ExternalRef>;
  createInvoice(input: InvoiceInput): Promise<ExternalRef>;
  recordPayment(input: PaymentInput): Promise<ExternalRef>;
  createCreditNote(input: CreditNoteInput): Promise<ExternalRef>;
  voidInvoice(externalInvoiceId: string): Promise<void>;
  listAccounts(): Promise<ExternalAccount[]>;
  listTaxRates(): Promise<ExternalTaxRate[]>;
}

export interface ExternalRef {
  externalId: string;
  raw?: unknown;
}

export interface ContactInput {
  name: string;
  email?: string;
  phone?: string;
  address?: { line1?: string; city?: string; region?: string; postcode?: string;
country?: string };
  externalIdHint?: string; // for upsert
}

export interface InvoiceLine {
  description: string;
  quantity: number;
  unitAmountCents: number;
  accountCode: string;
}
```

```

    taxCode?: string;
    taxRate?: number;
  }

  export interface InvoiceInput {
    contactExternalId: string;
    invoiceNumber: string;           // Drive247-issued number, never auto
    issueDate: string;               // YYYY-MM-DD
    dueDate?: string;
    currency: string;
    reference?: string;              // e.g. rental booking ref
    lines: InvoiceLine[];
  }

  export interface PaymentInput {
    invoiceExternalId: string;
    amountCents: number;
    currency: string;
    paidAt: string;                  // YYYY-MM-DD
    paymentAccountCode: string;
    reference?: string;
  }

  export interface CreditNoteInput {
    invoiceExternalId: string;
    amountCents: number;
    currency: string;
    issueDate: string;
    reason?: string;
  }

```

7.2 Two implementations

```

supabase/functions/_shared/accounting/
├─ types.ts
├─ xero-client.ts           // implements AccountingProvider for Xero
├─ zoho-client.ts           // implements AccountingProvider for Zoho
└─ factory.ts               // getProvider(tenant_id, provider) → AccountingProvider

```

`factory.ts` reads the active `accounting_connections` row, refreshes the access token if needed, and returns the correctly initialised client.

7.3 Idempotency contract

Every call must include an idempotency key on the wire when supported:

- **Xero:** use the `Idempotency-Key` header (supported on `POST /Invoices`, `/Payments`, `/CreditNotes`).
- **Zoho:** Zoho lacks a native header — instead, we check `accounting_contact_links` and `financial_event_sync_state.external_invoice_id` **before** every write, and short-circuit if a record already exists.

8. Event-to-Provider Mapping

8.1 The canonical mapping table

Drive247 event	Xero action	Zoho action	Notes
rental_charge	Invoice line	Invoice line	One invoice per rental, lines added as charges accumulate
extension_charge	New invoice (linked via reference)	New invoice (linked via reference)	Extensions get their own invoice per [feature_extensions]
damage_charge	Invoice line	Invoice line	Added to active rental invoice if open, else new
mileage_charge	Invoice line	Invoice line	Same as above
late_fee	Invoice line	Invoice line	Same as above
insurance_charge	Invoice line	Invoice line	Mapped to Insurance income account
charging_cost	Invoice line	Invoice line	Tesla Supercharger pass-through
deposit_capture	Invoice (separate, type "Customer Deposit")	Invoice flagged as deposit	Only on capture — preauths never sync
security_hold_release	No-op	No-op	Preauth release doesn't touch books
refund	Credit note + applied payment	Credit note + refund record	Linked back to source invoice
discount	Invoice line with negative amount	Invoice line with negative amount	Mapped to Discounts Given account
partner_payout	Bill (Phase 2)	Bill (Phase 2)	Out of scope for MVP
maintenance_expense	Bill (Phase 2)	Bill (Phase 2)	Out of scope for MVP

8.2 Default account & tax mapping (shipped out-of-box)

Event type	Default Xero account	Default Zoho category	Default tax (UK)	Default tax (US)
rental_charge	200 – Sales	Sales	OUTPUT2 (20%)	None
insurance_charge	260 – Other Revenue	Other Income	OUTPUT2	None
damage_charge	260 – Other Revenue	Other Income	OUTPUT2	None

mileage_charge	260 – Other Revenue	Other Income	OUTPUT2	None
late_fee	260 – Other Revenue	Other Income	NONE	None
charging_cost	200 – Sales	Sales	OUTPUT2	None
deposit_capture	260 – Other Revenue	Other Income	NONE	None
refund	(credit note — uses original line accounts)	(credit note)	(mirrors invoice)	(mirrors invoice)
discount	200 – Sales (negative)	Sales (negative)	OUTPUT2	None

Tenants can override any of these via the Mapping UI (Section 13).

8.3 Rental-to-invoice grouping rule

ONE invoice per rental_id, until:

- rental status = 'closed' → invoice finalised, no further lines accepted
- extension created → new invoice with reference "ORIG-INV-### / EXT-1"
- manual void → invoice voided, lines moved to new draft

This avoids creating dozens of micro-invoices per rental and matches how a human bookkeeper would do it.

9. Edge Functions Reference

All functions live under `supabase/functions/` and follow the existing project pattern (CORS helper, JSON response helper, service-role client). JWT enforcement is configured in `supabase/config.toml`.

Function	JWT	Trigger	Purpose
xero-oauth-start	yes	Portal button click	Generates auth URL, persists nonce
xero-oauth-callback	no	Xero redirect	Exchanges code → tokens, upserts connection
zoho-oauth-start	yes	Portal button click	Same as Xero
zoho-oauth-callback	no	Zoho redirect	Same as Xero
disconnect-accounting	yes	Portal disconnect button	Sets status = revoked
refresh-accounting-tokens	yes (cron)	Every 10 min	Refreshes expiring tokens
list-accounting-accounts	yes	Mapping UI load	Proxies listAccounts()
list-accounting-tax-rates	yes	Mapping UI load	Proxies listTaxRates()
save-accounting-mappings	yes	Mapping UI save	Upserts accounting_account_mappings

enqueue-financial-event	yes	Internal RPC from app code	Inserts financial_events + initial sync_state
process-accounting-sync	yes (cron)	Every 2 min	Picks pending sync_state rows and syncs
retry-accounting-sync	yes	Portal manual retry	Re-queues a single failed sync_state
backfill-accounting-sync	yes	Portal backfill UI	Spawns async backfill job
get-accounting-sync-status	yes	Portal sync log	Returns paginated sync_state with joins
get-vehicle-profitability	yes	Profitability dashboard	Aggregates revenue/expense per vehicle

9.1 process-accounting-sync — the heart of the system

Pseudocode (one tick of the cron):

```
const batch = await db.query(`
  select s.*, e.*
  from financial_event_sync_state s
  join financial_events e on e.id = s.financial_event_id
  where s.state in ('pending', 'failed')
     and (s.next_attempt_at is null or s.next_attempt_at <= now())
  order by s.next_attempt_at nulls first
  limit 100
  for update skip locked
`);

for (const row of batch) {
  await db.update(syncState, row.id, { state: 'syncing', last_attempt_at: now() });
  try {
    const provider = await getProvider(row.tenant_id, row.provider);

    const contactExternalId = await ensureContact(provider, row);
    const invoice           = await ensureInvoice(provider, row,
contactExternalId);

    if (row.event_type === 'refund') {
      const creditNote = await provider.createCreditNote({ ... });
      await persistSynced(row, { external_credit_note_id: creditNote.externalId });
    } else if (isPaymentEvent(row)) {
      const payment = await provider.recordPayment({ ... });
      await persistSynced(row, { external_payment_id: payment.externalId,
external_invoice_id: invoice.externalId });
    } else {
      await persistSynced(row, { external_invoice_id: invoice.externalId });
    }
  }
}
```



```

    } catch (err) {
      await persistFailure(row, err); // increments attempts, sets next_attempt_at
with backoff
    }
  }
}

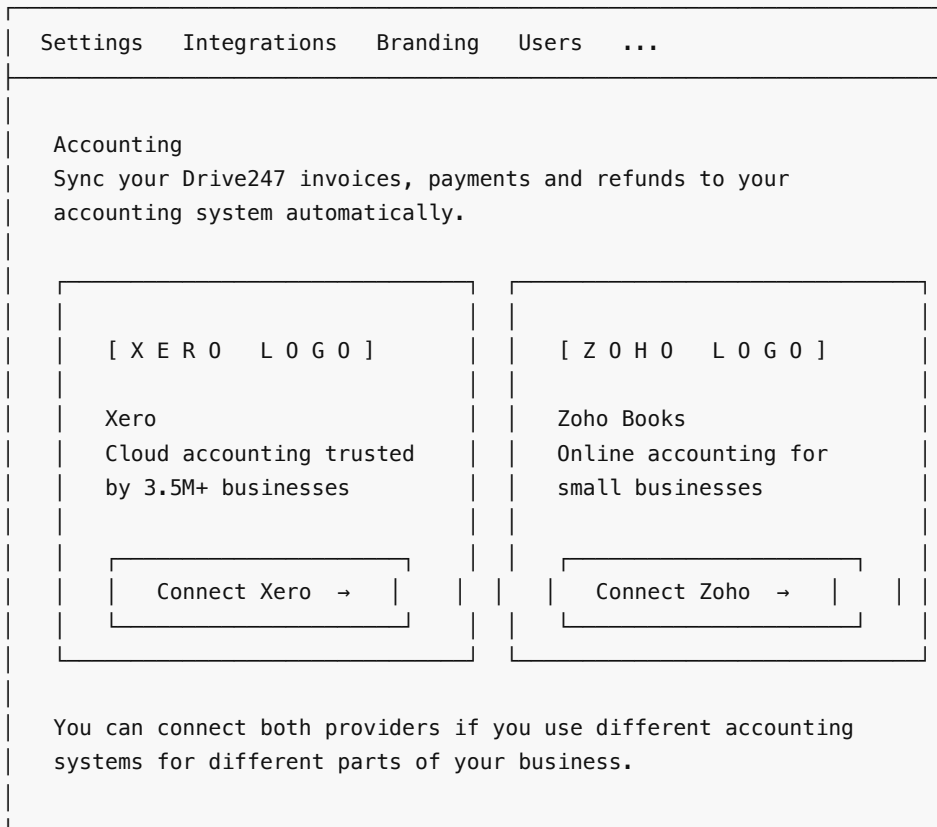
```

Backoff schedule: 1m, 5m, 30m, 2h, 12h, dead-letter . After dead-letter, the row stays **failed** and only manual retry from the UI will re-queue it.

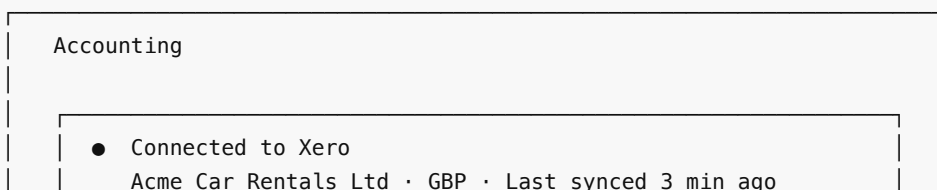
10. Portal UI — Screens & Specifications

All screens follow the Drive247 design system: DM Sans, indigo #6366f1 accent, flat surfaces, 1px borders.

10.1 Settings → Integrations → Accounting (entry point)



10.2 Connected state



[Configure mappings] [View sync log] [Disconnect]

○ Not connected to Zoho Books

[Connect Zoho]

10.3 OAuth — provider chooser (Zoho only)

Zoho requires picking the region before redirect:

Which Zoho region is your account in?

- ☒ Global (.com) United States
- ☐ Europe (.eu) UK / EU
- ☐ India (.in) India
- ☐ Australia (.com.au) Australia
- ☐ Japan (.jp) Japan

[Cancel] [Continue →]

10.4 Mapping screen — Settings → Integrations → Accounting → Configure mappings

← Configure mappings — Xero

Tell Drive247 which Xero account each type of charge should go to, and which tax rate to apply.

EVENT TYPE	XERO ACCOUNT	TAX RATE
Rental charge	[200 – Sales ▼]	[VAT 20% ▼]
Insurance charge	[260 – Other Rev ▼]	[VAT 20% ▼]
Damage charge	[260 – Other Rev ▼]	[VAT 20% ▼]
Mileage charge	[260 – Other Rev ▼]	[VAT 20% ▼]
Late fee	[260 – Other Rev ▼]	[No VAT ▼]
Charging cost	[200 – Sales ▼]	[VAT 20% ▼]
Deposit (captured)	[260 – Other Rev ▼]	[No VAT ▼]
Refund	(uses source invoice accounts)	
Discount	[200 – Sales ▼]	[VAT 20% ▼]

Payment account

When Drive247 records a payment in Xero, which bank/clearing account should it post against?

[090 – Stripe Clearing ▼]

[Cancel] [Save mappings]

10.5 Sync log — Settings → Integrations → Accounting → View sync log

← Sync log – Xero

Synced 2,847	Pending 12	Failed 3	Total 2,862
-----------------	---------------	-------------	----------------

Filter: [All ▼] [Last 30 days ▼] [Search invoice #] 🔍

EVENT	RENTAL	AMOUNT	STATUS	TIME	ACTION
Rental charge	R-1042	£450	✓ Synced	3 min ago	View
Refund	R-1037	–£75	✓ Synced	12 min ago	View
Damage charge	R-1031	£150	✗ Failed	1 hr ago	Retry
Late fee	R-1029	£25	⌚ Pending	Just now	–
Rental charge	R-1028	£820	✓ Synced	2 hr ago	View

< 1 2 3 4 5 ... 47 >

10.6 Failed-row detail drawer

Opens from "Retry" / row click on a failed sync:

Sync failed – Damage charge · R-1031

Event ID evt_8h2k...
Provider Xero
Attempts 5
Last attempted 26 May 2026, 14:32

Error

ValidationException: Account code "260" is not

active in this organisation.

Likely fix

→ Open Configure mappings and choose an active Xero account for "Damage charge".

[Open mappings] [Retry now] [Mark skipped]

10.7 Per-rental sync badges (on rental detail page)

Existing rental detail page gains a "Accounting" stripe under the Payments section:

Payments

...

— Accounting —

Xero ✓ Invoice INV-04127 · Synced 12 May 2026

 ✓ Payment £450 · Synced 12 May 2026

 ⌚ Damage charge £150 · Pending

Zoho ✓ Invoice INV-2026-031 · Synced 12 May 2026

10.8 Backfill wizard

A 3-step modal (Settings → Integrations → Accounting → "Sync historical data"):

Sync historical data to Xero

Step 1 of 3

Which date range should we sync?

☒ All time

☐ Last 12 months

☐ Custom: [01 Jan 2026] to [26 May 2026]

What's included

• 1,284 invoices

• 1,102 payments

• 78 refunds

• Estimated time: ~22 minutes

[Cancel] [Next →]

Steps 2 and 3 are: (2) confirm mappings are set, (3) review + start.

11. Vehicle Profitability Dashboard

A new sidebar entry under **Reports → Vehicle Profitability**. Independent of any accounting connection — driven entirely by `financial_events` .

11.1 Screen layout

Reports / Vehicle Profitability

Period [Last 12 months ▼] Currency GBP Compare [Off ▼]

Revenue
£284,210

Expenses
£87,420

Net Profit
£196,790

Avg ROI
42 %

VEHICLE		REVENUE	EXPENSES	PROFIT	UTIL	ROI	
Tesla M3	AB12	£42,180	£8,200	£33,980	78 %	62 %	★
Tesla MY	XY99	£38,440	£9,100	£29,340	81 %	58 %	
BMW 3-S	LK77	£12,300	£6,800	£5,500	44 %	18 %	
Audi A4	MN12	£4,800	£5,200	-£400	22 %	-2 %	×
...							

★ Top performer

× Loss-making

Clicking a row drills into per-vehicle breakdown: revenue by event type, monthly trend, maintenance log.

11.2 Underlying query

```
select
  v.id as vehicle_id,
  v.reg_number,
  v.make || ' ' || v.model as name,

  sum(case when e.event_type in ('rental_charge','extension_charge','damage_charge',
                                'mileage_charge','late_fee','charging_cost',
                                'insurance_charge')
        then e.amount_cents else 0 end) / 100.0 as revenue,

  sum(case when e.event_type in ('maintenance_expense','partner_payout')
        then e.amount_cents else 0 end) / 100.0 as expenses,

  sum(case when e.event_type = 'refund'
        then e.amount_cents else 0 end) / 100.0 as refunds,
```

```

    (sum(case when e.event_type in
('rental_charge','extension_charge','damage_charge',
                                'mileage_charge','late_fee','charging_cost',
                                'insurance_charge')
        then e.amount_cents else 0 end)
    - sum(case when e.event_type in ('maintenance_expense','partner_payout','refund')
        then e.amount_cents else 0 end)
    ) / 100.0 as net_profit

from vehicles v
left join financial_events e
  on e.vehicle_id = v.id
  and e.occurred_at between $1 and $2
where v.tenant_id = $3
group by v.id, v.reg_number, v.make, v.model
order by net_profit desc;

```

Utilisation comes from a separate `rentals` aggregate.

12. Historical Backfill

12.1 Strategy

When a tenant connects a provider, the entire pre-connection history is **not** synced automatically — that would create thousands of duplicate invoices in their accountant's system. Instead, the tenant opts in via the Backfill wizard.

12.2 Backend job

1. UI calls `backfill-accounting-sync` with `{ provider, date_from, date_to }`.
2. Function inserts a row into a new `backfill_jobs` table and returns immediately.
3. A cron-triggered `process-backfill-jobs` worker:
 - Reads pending events in the range that have no `sync_state` row for the provider.
 - Inserts `financial_event_sync_state` rows with `state = 'pending'`.
 - The normal `process-accounting-sync` cron picks them up.
4. Backfill UI polls `get-accounting-sync-status?backfill_job_id=...` for progress.

12.3 Rate-limit awareness

The backfill worker respects provider rate limits:

- **Xero:** max 50 calls/min (leaving headroom under the 60/min limit).
- **Zoho:** max 80 calls/min.

The `process-accounting-sync` cron picks up at most 100 events per tick and stays well under both ceilings.

13. Account & Tax Mapping

13.1 What gets mapped

Mapping	Source	Target
Event type → income account	financial_event_type enum	accounting_account_mappings.external_account_code
Event type → tax rate	financial_event_type enum	accounting_account_mappings.external_tax_code
Payment → bank account	tenant-level setting	accounting_account_mappings row with event_type = 'payment_account' (sentinel)

13.2 First-connect UX

Immediately after the OAuth round-trip succeeds we redirect to the Mapping screen pre-populated with the **defaults from Section 8.2**, plus a friendly banner:

"We've suggested defaults based on a typical car rental business. Review and save — you can change these any time."

The tenant cannot skip this step on first connect (Save is mandatory). After that, the screen is editable from Settings.

13.3 Account list discovery

Calling `list-accounting-accounts` proxies the provider:

- **Xero:** GET `/Accounts?where=Status=="ACTIVE"`
- **Zoho:** GET `/chartofaccounts`

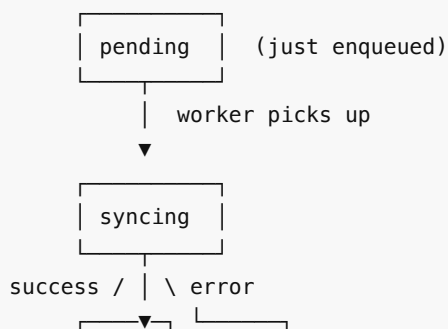
Cached in-memory per-tenant for 5 minutes. A "Refresh" button next to each dropdown forces re-fetch.

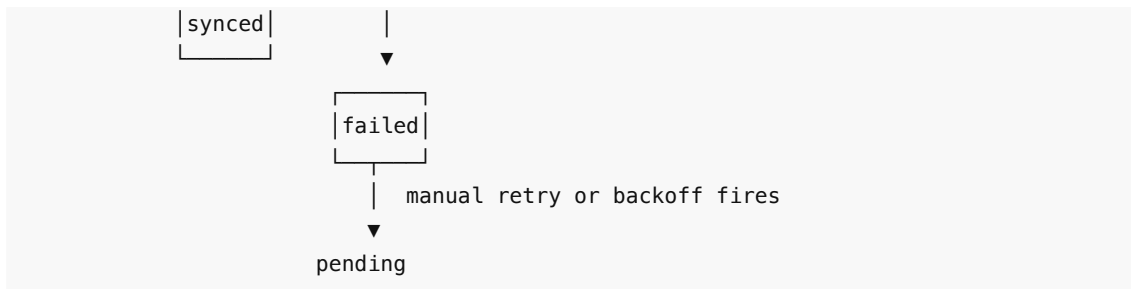
13.4 Tax rate discovery

- **Xero:** GET `/TaxRates` — returns codes like `OUTPUT2`, `INPUT2`, `NONE`, plus jurisdictional rates for non-UK orgs.
- **Zoho:** GET `/settings/taxes` — returns per-org UUIDs; persist both `external_tax_code` (id) and `external_tax_rate` (percent) for display.

14. Sync Status, Logs & Error Handling

14.1 State machine





`skipped` is a manual override (operator marks an event as "do not sync"), reachable from the failed-row drawer.

14.2 Error classification

Error class	Examples	Auto-retry?	Surface to user?
Transient	429, 5xx, network timeout	Yes	After 3rd failure
Auth	401, refresh failed	No (mark connection expired)	Yes, prominent banner
Validation	Invalid account code, missing tax	No	Yes, detail drawer with fix link
Duplicate	Already synced (idempotency hit)	No	No (silent success)
Unknown	Anything else	Yes, 3 attempts max	After 3rd failure

14.3 In-app notifications

When a connection expires or a sync has been failing for >24h:

- Insert a row into the existing `reminders` table with `severity = 'warning'`.
- Show a top banner in the portal:

"Your Xero connection has expired. Reconnect to keep syncing. [Reconnect →]"

15. Webhooks (Phase 2)

Not in scope for MVP. Documented here so the schema is forward-compatible.

15.1 Xero

- Single webhook endpoint registered in the developer portal.
- Signed with `x-xero-signature` (HMAC-SHA256 of body using webhook key).
- Events: `INVOICE.UPDATE`, `CONTACT.UPDATE`.
- On `INVOICE.UPDATE` with status `PAID`, mark the corresponding `financial_event_sync_state.external_invoice_paid_at`.

15.2 Zoho

- Configured per-org as **workflow rules** inside Zoho Books (the tenant has to set this up — we provide a one-click "Install workflow" button that calls Zoho's automation API).
- Signed with `x-zoho-webhook-signature` .
- Events: `invoice.thresholds.paid` , `customerpayment.created` .

15.3 Database additions (deferred)

```
alter table financial_event_sync_state
  add column external_invoice_paid_at timestamptz,
  add column external_status text;
```

16. Testing Strategy

16.1 Manual test matrix (per provider)

Scenario	Steps	Expected
Connect	Click Connect → OAuth → return	<code>accounting_connections.status = 'active'</code> , redirect to mapping screen
Disconnect	Click Disconnect → confirm	<code>status = 'revoked'</code> , no further syncs run
Token refresh	Set <code>token_expires_at</code> to past, trigger cron	New <code>access_token</code> persisted, sync resumes
New rental charge	Create rental in Drive247	<code>financial_events</code> row created, <code>sync_state</code> goes pending → synced, invoice appears in provider
Extension	Add extension to closed-invoice rental	New invoice in provider, original untouched
Refund	Refund a payment in Drive247	Credit note appears in provider, linked to original invoice
Mapping change	Change "Damage charge" → different account	New damage charges use new account; existing invoices unchanged
Failed sync — invalid account	Map to deleted Xero account	Sync fails, drawer shows fix-link, retry after fix succeeds
Backfill	Connect on Day N, run backfill for Day N-30 → N	All historical events appear in provider with correct dates
Connection expired	Manually set <code>status = 'expired'</code>	Banner shows, no new syncs run, reconnect flow works
Dual provider	Connect both Xero and Zoho	Each event has 2 <code>sync_state</code> rows, both succeed independently

16.2 Automated tests

```
apps/portal/src/__tests__/hooks/use-accounting-connection.test.ts
apps/portal/src/__tests__/hooks/use-financial-events.test.ts
apps/portal/src/__tests__/components/accounting-mapping-form.test.tsx
supabase/functions/_shared/accounting/__tests__/xero-client.test.ts
supabase/functions/_shared/accounting/__tests__/zoho-client.test.ts
supabase/functions/_shared/accounting/__tests__/sync-worker.test.ts
```

- Xero/Zoho HTTP calls mocked via `msw` or `vitest`'s `fetch` mock.
- Worker tested by inserting fixture `financial_events` and asserting `sync_state` transitions.

16.3 End-to-end smoke test

A Vitest script that:

1. Creates a tenant + connects to a Xero Demo Company (using a pre-issued long-lived refresh token in CI).
2. Inserts a `rental_charge` event.
3. Polls Xero's `GET /Invoices` until the new invoice appears.
4. Tears down (voids the invoice in Xero).

Runs nightly, not on every PR (Xero rate limits).

17. Phased Build Sequence

Estimated by an experienced full-stack developer working in this codebase, allowing for code review and QA between sprints.

Sprint 1 — Ledger foundation (5 days)

- ☐ Migration: `financial_events`, `financial_event_sync_state`
- ☐ Trigger or app-level RPCs to populate `financial_events` from existing `rental_charges`, `payments`, `rental_extensions`, `refunds`
- ☐ Unit tests for ledger writes

Sprint 2 — Xero OAuth + connection (5 days)

- ☐ Migration: `accounting_connections`, `accounting_contact_links`, `oauth_state`
- ☐ Edge functions: `xero-oauth-start`, `xero-oauth-callback`, `disconnect-accounting`, `refresh-accounting-tokens`
- ☐ Portal UI: Settings → Integrations → Accounting (entry + connected states)
- ☐ Manual test: Connect → Disconnect → Reconnect

Sprint 3 — Xero sync engine + mappings (8 days)

- ☐ Migration: `accounting_account_mappings`
- ☐ `_shared/accounting/types.ts` + `xero-client.ts`
- ☐ Edge functions: `enqueue-financial-event`, `process-accounting-sync`, `list-accounting-accounts`, `list-accounting-tax-rates`, `save-accounting-mappings`, `retry-accounting-sync`
- ☐ Portal UI: Mapping screen, Sync log, failed-row drawer
- ☐ Per-rental sync badges on rental detail page

Sprint 4 — Backfill + Profitability dashboard (6 days)

- ☐ Migration: `backfill_jobs`
- ☐ Edge functions: `backfill-accounting-sync` , `process-backfill-jobs` , `get-vehicle-profitability`
- ☐ Portal UI: Backfill wizard, Profitability dashboard page
- ☐ End-to-end smoke test wired up in CI

Sprint 5 — Zoho parity (6 days)

- ☐ `_shared/accounting/zoho-client.ts`
- ☐ Edge functions: `zoho-oauth-start` , `zoho-oauth-callback` (+ region selector)
- ☐ Portal UI: Zoho card on entry screen, region selector modal
- ☐ Sync engine: provider abstraction wired so Zoho works through the same `process-accounting-sync`
- ☐ Manual test matrix run against Zoho

Sprint 6 — Hardening & launch (4 days)

- ☐ Rate-limit guards, backoff verification
- ☐ In-app notification banners for expired connections
- ☐ Documentation pass (operator-facing help doc + this guide)
- ☐ Production credentials handover, environment variable population
- ☐ Soft launch to first 3 tenants

Total: 34 working days (~7 calendar weeks for one developer).

18. Environment Variables

```
# Xero
XERO_CLIENT_ID=
XERO_CLIENT_SECRET=
XERO_REDIRECT_URI=
XERO_WEBHOOK_KEY= # Phase 2

# Zoho
ZOH0_CLIENT_ID=
ZOH0_CLIENT_SECRET=
ZOH0_REDIRECT_URI=
ZOH0_DEFAULT_DC=com # com | eu | in | com.au | jp | sa
ZOH0_WEBHOOK_KEY= # Phase 2

# Shared
ACCOUNTING_SYNC_BATCH_SIZE=100
ACCOUNTING_SYNC_RATE_LIMIT_XERO=50 # calls/min
ACCOUNTING_SYNC_RATE_LIMIT_ZOH0=80
ACCOUNTING_SYNC_DEAD_LETTER_AFTER=5 # attempts
```

These should be set in Supabase project secrets (not committed to the repo) and read inside edge functions via `Deno.env.get(...)` .

19. API Reference Cheat Sheet

19.1 Xero — endpoints used

Verb	URL	Purpose
GET	/connections	List tenants (orgs) the user authorised
GET	/Accounts	Chart of accounts
GET	/TaxRates	Tax rates for the org
POST	/Contacts	Upsert contact (use ContactNumber for idempotency)
POST	/Invoices	Create invoice (use InvoiceNumber for idempotency)
POST	/Payments	Record payment
POST	/CreditNotes	Create credit note
POST	/Invoices/{id} with Status=VOIDED	Void invoice

All calls require headers: `Authorization: Bearer <access_token>` , `Xero-tenant-id: <tenantId>` , `Accept: application/json` .

19.2 Zoho — endpoints used

Verb	URL	Purpose
GET	/organizations	List orgs the user has access to
GET	/chartofaccounts?organization_id=...	Chart of accounts
GET	/settings/taxes?organization_id=...	Tax rates
POST	/contacts?organization_id=...	Create contact
POST	/invoices?organization_id=...	Create invoice
POST	/customerpayments?organization_id=...	Record payment
POST	/creditnotes?organization_id=...	Create credit note
POST	/invoices/{id}/status/void?organization_id=...	Void invoice

All calls require headers: `Authorization: Zoho-oauthtoken <access_token>` , `Accept: application/json` . The `organization_id` query parameter is **mandatory** on every call.

19.3 Helpful response examples

A successful Xero invoice creation returns:

```
{
  "Invoices": [{
    "InvoiceID": "297c2dc5-cc47-4afd-8ec8-74990b8761e9",
    "InvoiceNumber": "INV-0001",
    "Status": "AUTHORISED",
    "Total": 450.00
  }]
}
```

A successful Zoho invoice creation returns:

```
{
  "code": 0,
  "message": "The invoice has been created.",
  "invoice": {
    "invoice_id": "460000000017369",
    "invoice_number": "INV-2026-031",
    "status": "sent",
    "total": 450.00
  }
}
```

20. Appendix — Glossary & Useful Links

20.1 Glossary

Term	Meaning
Tenant	A rental operator using Drive247 (multi-tenant model)
Provider	The external accounting system: Xero or Zoho Books
Connection	One tenant's authenticated link to one provider
Financial event	A row in <code>financial_events</code> — anything chargeable, refundable, or accountable
Sync state	The state of one event against one provider
External ref	An identifier issued by the provider (e.g. Xero InvoiceID)
Mapping	A tenant-configured rule for which provider account/tax to use for each event type
Backfill	One-off operation to sync historical events that pre-date the connection
Dead letter	An event that exceeded retry budget; only manual retry resurrects it

20.2 Useful links

- Xero API docs: <https://developer.xero.com/documentation/api/accounting/overview>
- Xero OAuth 2.0: <https://developer.xero.com/documentation/guides/oauth2/overview>
- Xero rate limits: <https://developer.xero.com/documentation/guides/oauth2/limits>

- **Xero TaxType reference:** <https://developer.xero.com/documentation/api/accounting/types#tax-types>
- **Xero Idempotency:** <https://developer.xero.com/documentation/guides/how-to-guides/idempotency>
- **Zoho Books API:** <https://www.zoho.com/books/api/v3/>
- **Zoho OAuth 2.0:** <https://www.zoho.com/accounts/protocol/oauth.html>
- **Zoho data centres:** <https://www.zoho.com/crm/developer/docs/api/v3/multi-dc.html>
- **Zoho rate limits:** <https://www.zoho.com/books/api/v3/introduction/#rate-limit>

End of guide. Questions, edge cases, or scope changes should be raised against this document so we keep one source of truth. Update the version line below on every meaningful change.

Version: 1.0 — May 2026